

CFD Bonus Assignment

Ambuj Nayan
24M0003

Aerospace Engineering
Indian Institute of Technology Bombay

I. PROBLEM STATEMENT

Consider a physical domain between an airfoil (NACA 63-412) and far field boundary located at 10m as shown in the Figure 1. A structured computational grid of size 40×40 (i.e., 40 points in both ξ and η directions) is generated.

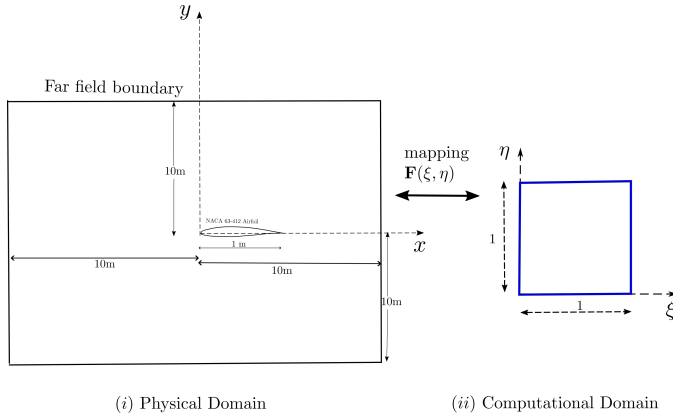


Fig. 1: Problem Statement

The physical grid is to be first generated using **Transfinite Interpolation (TFI)**, which provides an initial structured grid by interpolating between boundary curves. This grid is then should be refined using **Elliptic Grid Generation** to improve smoothness.

II. IMPLEMENTATION

The computational coordinates are defined as follows:

- $\xi \in [0, 1]$ parameterizes the points along the inner (airfoil) and outer (far-field) boundaries.
- $\eta \in [0, 1]$ controls the interpolation from the airfoil surface ($\eta = 0$) to the far-field boundary ($\eta = 1$), which is taken as a square enclosing the airfoil.

A. Constructing Airfoil

- Airfoil Point data from .csv file was extracted.
- The extracted x and y values were divided by 100, effectively normalizing the airfoil chord to unity. This normalization is crucial for compatibility with grid generation and flow solvers.
- For smooth surface representation and to generate a refined set of airfoil points, cubic spline interpolation was used. The x and y coordinates were first parameterized over a linearly spaced variable $t \in [0, 1]$.

- The `CubicSpline` class from `scipy.interpolate` was used to interpolate both $x(t)$ and $y(t)$ separately, using clamped boundary conditions. This ensures continuity of both first and second derivatives at the endpoints, helping preserve curvature at the leading and trailing edges.
- This densely sampled airfoil contour was used as the inner boundary for the transfinite interpolation grid.

B. TFI

The mapping function $\mathbf{F}(\xi, \eta) = (x(\xi, \eta), y(\xi, \eta))$ is constructed using transfinite interpolation.

Physical domain:

- **Inner boundary (airfoil):**

$$x_{\text{inner}} = x_{\text{airfoil}}, \quad y_{\text{inner}} = y_{\text{airfoil}}$$

- **Outer boundary (Square):** The square boundary is defined with edge length 20, centered at the origin, with points generated counterclockwise:

- For $\xi \in [0, 0.125]$: Upper right edge (10, 0) to (10, 10)
- For $\xi \in [0.125, 0.375]$: Top edge (10, 10) to (-10, 10)
- For $\xi \in [0.375, 0.625]$: Left edge (-10, 10) to (-10, -10)
- For $\xi \in [0.625, 0.875]$: Bottom edge (-10, -10) to (10, -10)
- For $\xi \in [0.875, 1]$: Lower right edge (10, -10) to (10, 0)

Computational Domain:

$$\mathbf{F}(\xi, \eta) = \begin{bmatrix} x(\xi, \eta) \\ y(\xi, \eta) \end{bmatrix}$$

- **Corner Points:**

$$\mathbf{F}(0, 0) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{F}(1, 0) = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

$$\mathbf{F}(0, 1) = \begin{bmatrix} 10 \\ 0 \end{bmatrix}, \quad \mathbf{F}(1, 1) = \begin{bmatrix} 10 \\ 0 \end{bmatrix}$$

- **Edges:**

$$\mathbf{F}(\xi, 0) = \begin{bmatrix} x_{\text{airfoil}} \\ y_{\text{airfoil}} \end{bmatrix}, \quad \mathbf{F}(0, \eta) = \begin{bmatrix} 1 - \eta + 10\eta \\ 0 \end{bmatrix}$$

$$\mathbf{F}(\xi, 1) = \begin{bmatrix} x_{square} \\ y_{square} \end{bmatrix}, \quad \mathbf{F}(1, \eta) = \begin{bmatrix} 1 - \eta + 10\eta \\ 0 \end{bmatrix}$$

Using TFI, the mapping function is expressed as:[1]

$$\begin{aligned} P_\xi[\mathbf{F}(\eta)] &= (1 - \xi)\mathbf{F}(0, \eta) + \xi\mathbf{F}(1, \eta) \\ &= (1 - \xi) \begin{bmatrix} 1 + 9\eta \\ 0 \end{bmatrix} + \xi \begin{bmatrix} 1 + 9\eta \\ 0 \end{bmatrix} \end{aligned}$$

$$\begin{aligned} P_\eta[\mathbf{F}(\xi)] &= (1 - \eta)\mathbf{F}(\xi, 0) + \eta\mathbf{F}(\xi, 1) \\ &= (1 - \eta) \begin{bmatrix} x_{airfoil} \\ y_{airfoil} \end{bmatrix} + \eta \begin{bmatrix} x_{square} \\ y_{square} \end{bmatrix} \end{aligned}$$

$$\begin{aligned} P_\xi P_\eta(\mathbf{F}) &= (1 - \xi)(1 - \eta)\mathbf{F}(0, 0) + (1 - \xi)\eta\mathbf{F}(0, 1) \\ &\quad + \xi(1 - \eta)\mathbf{F}(1, 0) + \xi\eta\mathbf{F}(1, 1) \\ &= (1 - \xi)(1 - \eta) \begin{bmatrix} 1 \\ 0 \end{bmatrix} + (1 - \xi)\eta \begin{bmatrix} 10 \\ 0 \end{bmatrix} \\ &\quad + \xi(1 - \eta) \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \xi\eta \begin{bmatrix} 10 \\ 0 \end{bmatrix} \end{aligned}$$

TFI Expression:[1]

$$\begin{aligned} \mathbf{F}(\xi, \eta) &= P_\xi[\mathbf{F}(\eta)] \oplus P_\eta[\mathbf{F}(\xi)] \\ &= P_\xi[\mathbf{F}(\eta)] + P_\eta[\mathbf{F}(\xi)] - P_\xi P_\eta(\mathbf{F}) \end{aligned}$$

Substituting we get:

$$\begin{aligned} \mathbf{F}(\xi, \eta) &= (1 - \xi) \begin{bmatrix} 1 + 9\eta \\ 0 \end{bmatrix} + \xi \begin{bmatrix} 1 + 9\eta \\ 0 \end{bmatrix} \\ &\quad + (1 - \eta) \begin{bmatrix} x_{airfoil} \\ y_{airfoil} \end{bmatrix} + \eta \begin{bmatrix} x_{square} \\ y_{square} \end{bmatrix} \\ &\quad - (1 - \xi)(1 - \eta) \begin{bmatrix} 1 \\ 0 \end{bmatrix} - (1 - \xi)\eta \begin{bmatrix} 10 \\ 0 \end{bmatrix} \\ &\quad - \xi(1 - \eta) \begin{bmatrix} 1 \\ 0 \end{bmatrix} - \xi\eta \begin{bmatrix} 10 \\ 0 \end{bmatrix} \end{aligned} \quad (1)$$

Simplyfying, we get:

$$\mathbf{F}(\xi, \eta) = \begin{bmatrix} x(\xi, \eta) \\ y(\xi, \eta) \end{bmatrix} = (1 - \eta) \begin{bmatrix} x_{airfoil} \\ y_{airfoil} \end{bmatrix} + \eta \begin{bmatrix} x_{square} \\ y_{square} \end{bmatrix}$$

$$\mathbf{x}(\xi, \eta) = (1 - \eta)x_{airfoil} + \eta x_{square}$$

$$\mathbf{y}(\xi, \eta) = (1 - \eta)y_{airfoil} + \eta y_{square}$$

Implementation Details:

- The square boundary is generated using piecewise linear interpolation
- $\eta = 0$ corresponds to the airfoil surface
- $\eta = 1$ corresponds to the square boundary

C. Elliptic Grid Generation

[2]

$$\begin{aligned} x_{i,j} &= \left\{ \frac{a}{(\Delta\xi)^2} [x_{i+1,j} + x_{i-1,j}] + \frac{c}{(\Delta\eta)^2} [x_{i,j+1} + x_{i,j-1}] \right. \\ &\quad \left. - \frac{b}{2\Delta\xi\Delta\eta} [x_{i+1,j+1} - x_{i+1,j-1} + x_{i-1,j-1} - x_{i-1,j+1}] \right\} \\ &\quad / \left\{ 2 \left[\frac{a}{(\Delta\xi)^2} + \frac{c}{(\Delta\eta)^2} \right] \right\} \\ y_{i,j} &= \left\{ \frac{a}{(\Delta\xi)^2} [y_{i+1,j} + y_{i-1,j}] + \frac{c}{(\Delta\eta)^2} [y_{i,j+1} + y_{i,j-1}] \right. \\ &\quad \left. - \frac{b}{2\Delta\xi\Delta\eta} [y_{i+1,j+1} - y_{i+1,j-1} + y_{i-1,j-1} - y_{i-1,j+1}] \right\} \\ &\quad / \left\{ 2 \left[\frac{a}{(\Delta\xi)^2} + \frac{c}{(\Delta\eta)^2} \right] \right\} \end{aligned}$$

Where a , b and c are:

$$a = x_\eta^2 + y_\eta^2$$

$$b = x_\xi x_\eta + y_\xi y_\eta$$

$$c = x_\xi^2 + y_\xi^2$$

III. RESULTS AND DISCUSSION

A. Airfoil

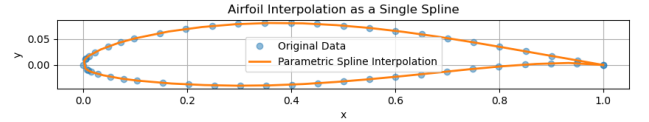


Fig. 2: Smooth airfoil surface generated by cubic spline interpolation from discrete point data. The resulting geometry ensures a continuous and smooth surface necessary for high-quality grid generation.

B. Comparison between TFI and Elliptic Grid

To assess the quality of mesh generation techniques, two approaches were implemented and compared: Transfinite Interpolation (TFI) and Elliptic Grid Generation. Each method was applied to the same domain bounded internally by the airfoil and externally by a square boundary.

a) Discussion::

- The TFI method rapidly produces a mesh with minimal computational overhead, but lacks control over grid smoothness and spacing. This is particularly problematic in high-gradient regions.

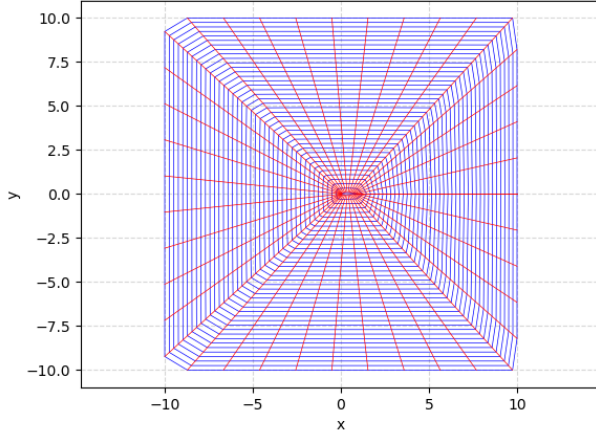


Fig. 3: Grid generated using Transfinite Interpolation. While fast and easy to implement, this method can produce poor grid quality near regions with high curvature, such as the trailing edge.

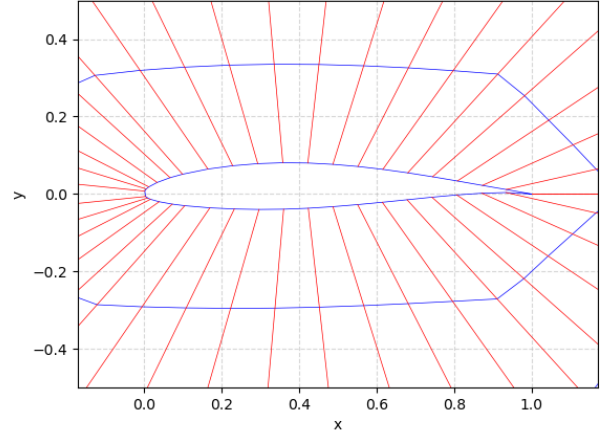


Fig. 5: Zoomed view of the TFI grid near the airfoil.

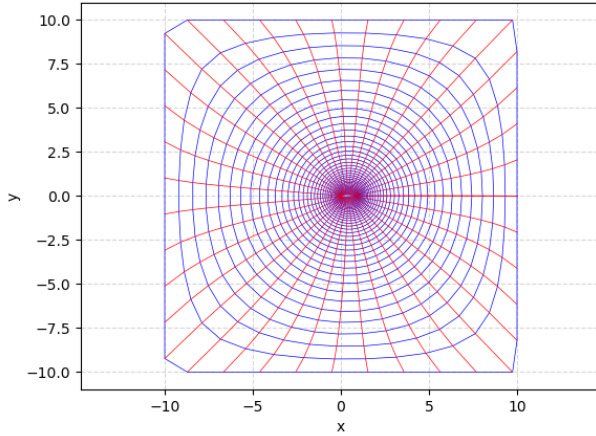


Fig. 4: Grid generated using Elliptic smoothing. The mesh exhibits improved orthogonality and smoothness, especially around the leading and trailing edges of the airfoil.

- The elliptic grid solves a PDE-based smoothing system that improves mesh quality by optimizing interior point distribution. This results in a grid that is both smoother and more orthogonal near the airfoil surface.
- While the elliptic method is computationally more intensive, the increase in mesh quality justifies its use, especially for flow simulations where numerical accuracy is sensitive to grid quality.
- Visual inspection shows that the elliptic grid offers better cell shape regularity, particularly in critical aerodynamic regions near the leading and trailing edges.

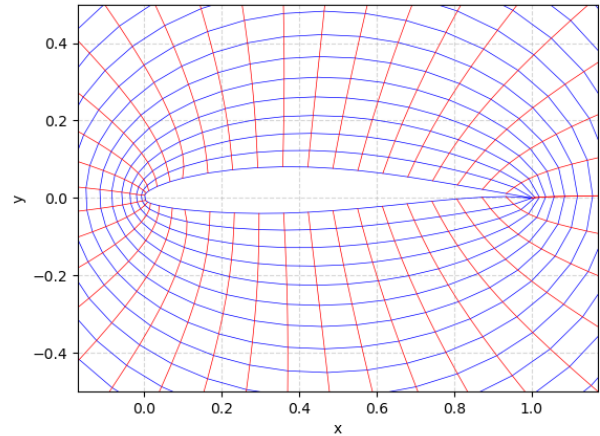


Fig. 6: Zoomed view of the elliptic grid near the airfoil. The grid lines are smoother and more aligned with the surface.

REFERENCES

- [1] T. J. Chung, *Computational Fluid Dynamics*. Cambridge University Press, 2010, ch. 17, pp. 555–561 (cit. on p. 2).
- [2] K. A. Hoffmann and S. T. Chiang, *Computational Fluid Dynamics Volume 1*. Engineering Education System, 2000, ch. 9, pp. 383–403 (cit. on p. 2).